Introduction

Load balancers are supposed to be boring. Requests come in, they get spread evenly across a fleet of servers, and nobody thinks about it again. In practice, the routing rule that decides *which* server gets a request isn't always fair — depending on timing and server speed, one machine can quietly end up handling far more traffic than its neighbors. At the scale of a real production system, you won't catch that by eyeballing a log.

This is exactly the scenario LeetCode 1606, *Find Servers That Handled Most Number of Requests*, is built around. It's a deceptively simple-sounding simulation problem that turns into a real test of whether you can recognize when a "just loop through it" solution is going to collapse under scale, and whether you know the right data structures to replace it with. Interviewers like this problem because it rewards exactly that instinct — noticing that a sorted list plus a heap beats a linear scan, and that the order you process events in can save you a sort entirely.

If you've ever designed a connection pool, a job queue, or a cache eviction policy, the core idea here — splitting "available right now" from "busy until some known time" — is a pattern you'll reuse constantly. That's the real reason to learn this problem properly instead of memorizing the solution.

Problem Overview

You have `k` servers, numbered `0` through `k - 1`. Requests arrive one at a time, and each request has an arrival time and a load (how long it takes to process). A server can only work on one request at a time.

For request `i`, the system first tries server `i mod k`. If that server is busy, it checks the next server in line, wrapping back around to server `0` if it runs past the last one. The first free server it finds gets the request. If every server is busy when request `i` arrives, that request is simply dropped — nobody handles it.

After all requests have arrived and been routed (or dropped), you need to report which server, or servers, ended up handling the most requests overall.

The two numbers that matter for difficulty: `k` and the number of requests can each be as large as 100,000. Any solution that checks servers one at a time for every single request is going to be far too slow.

Example

Take 3 servers, all free at the start, with 5 requests arriving in order.

- Request 0 → tries server `0 mod 3 = 0`. Server 0 is free. Assigned to server 0.
- Request 1 → tries server `1 mod 3 = 1`. Free. Assigned to server 1.
- Request 2 → tries server `2 mod 3 = 2`. Free. Assigned to server 2.
- Request 3 → tries server `3 mod 3 = 0`. Server 0 is still busy (its job hasn't finished yet), so it checks server 1, which is free. Assigned to server 1.
- Request 4 → tries server `4 mod 3 = 1`. Every server is currently busy. Request 4 is dropped.

Tallying it up: server 0 handled 1 request, server 1 handled 2 requests, server 2 handled 1 request. Server 1 is the busiest, having handled the most requests of the group.

Notice something important: the problem never tells you which server took which request. You have to replay the routing rule yourself, request by request, to figure that out before you can even start counting.

Intuition

The naive instinct, reading the problem statement, is "this is round robin — just keep a pointer and a busy/free boolean array." That instinct is wrong, and it's wrong in a way that's easy to miss until you trace through an example carefully.

The problem isn't just "is this server busy right now" — it's "is this server busy right now, *and if so, when does it stop being busy*." A boolean can't answer the second half. If server 0 started a long job 10 time units ago and finishes in 2 more, a simple busy flag has no way to know that "soon" is different from "forever." You need to track exact finish times.

Once you see that, the natural fix appears: keep two groups.

- **Available servers** — free right now, ready to take the next request.
- **Busy servers** — each tagged with the exact time its current job finishes.

Because requests arrive in strictly increasing time order, you can process them one at a time and, right before handling each new request, release any busy server whose finish time has already passed. No sorting required — the arrival order already gives you the order you need.

That gives you two jobs per request: 1. Free up any servers whose jobs are done. 2. Find the smallest available server number that is at or after `i mod k` (wrapping to the smallest available server overall if nothing qualifies).

A **min-heap** keyed by finish time handles job 1 efficiently — you always know who's about to free up next without scanning the whole busy set. A **sorted list** of available server numbers, searched with binary search (`bisect_left`), handles job 2 — finding the right starting point is a lookup instead of a walk through every server.

Brute Force Solution

Idea: For every request, walk through the servers one by one starting at `i mod k` , checking each one's busy/free state (by comparing to a tracked finish time) until a free one is found or you've checked all `k` servers.

Algorithm: 1. Keep a `finish_time` array, one entry per server, initialized to `-1` (free). 2. For each request `i` , starting at `i mod k` , scan forward (wrapping around) checking `finish_time[server] <= arrival[i]` . 3. If a free server is found, assign the request, update its finish time, and increment its count. 4. If no free server is found after checking all `k` servers, drop the request.

Advantages: Dead simple to write, obviously correct, easy to reason about.

Disadvantages: In the worst case, a single request can require checking all `k` servers. With both `k` and the number of requests up to 100,000, that's up to 10 billion checks — nowhere close to fast enough.

Complexity: Time `O(n * k)` in the worst case, space `O(k)` .

Optimal Solution

The optimal approach keeps two data structures instead of one flat array:

Structure	Holds	Purpose
`available` (sorted list)	Server indices currently free	Binary search for the smallest free server ≥ `i mod k`
`busy` (min-heap)	`(finish_time, server)` pairs	Know which servers free up next, without scanning all of them

For each request `i` with arrival time `arrival[i]` and load `load[i]` :

1. **Release finished servers.** Pop from `busy` while the top's finish time is `<= arrival[i]` , inserting each released server back into `available` in sorted order.
2. **Handle a full fleet.** If `available` is empty, drop the request and move to the next one.

3. **Find the right server.** Use `bisect_left(available, i % k)` to find the smallest available server at or after `i % k`. If the search runs off the end of the list, wrap to index `0` (the smallest available server overall).
4. **Assign.** Remove that server from `available`, increment its request count, and push `(arrival[i] + load[i], server)` onto `busy`.

This turns a per-request `O(k)` worst case into something dominated by heap operations (`O(log k)`) plus the sorted-list insert/search, which in Python's list-based implementation costs `O(k)` for the shift but is fast in practice for this problem's constraints — the key win is that you're no longer scanning *every* server on *every* request, only the ones relevant to freeing up and reassigning.

Python Code

```
import heapq
import bisect


class Solution:
    def busiestServers(self, k: int, arrival: list[int], load: list[int]) -> list[int]:
        available = list(range(k))          # free servers, kept sorted
        busy = []                           # heap of (finish_time, server)
        count = [0] * k

        for i, start in enumerate(arrival):
            # Release any servers that have finished by this request's arrival.
            while busy and busy[0][0] <= start:
                _, server = heapq.heappop(busy)
                bisect.insort(available, server)

            if not available:
                continue  # every server busy, request dropped

            # Find the smallest available server at or after i % k.
            pos = bisect.bisect_left(available, i % k)
            if pos == len(available):
                pos = 0  # wrap around to the smallest free server overall

            server = available.pop(pos)
            count[server] += 1
            heapq.heappush(busy, (start + load[i], server))

        best = max(count)
        return [i for i, c in enumerate(count) if c == best]
```

Code Walkthrough

- `available = list(range(k))` : every server starts free, stored in sorted order since `range(k)` is already ascending.
- `busy = []` : a heap of `(finish_time, server)` tuples; Python's `heapq` keeps the smallest `finish_time` at the top, so the next server to free up is always a peek away.
- `count = [0] * k` : running tally of requests handled per server.
- The `while busy and busy[0][0] <= start` loop: this must run **before** assignment, and it's the step that keeps a finished server from looking busy forever — `busy[0]` peeks the heap's minimum without popping, so we only pop when a server is actually done.
- `bisect.insort(available, server)` : re-inserts a freed server while keeping the list sorted, which is what makes the next `bisect_left` call valid.
- `bisect.bisect_left(available, i % k)` : this is the line that enforces the exact routing rule from the problem — search must start at `i % k` , not at `0` , or you've silently changed the problem.
- `if pos == len(available): pos = 0` : wraparound — if no available server index is `>= i % k` , the smallest available server overall (at the front of the sorted list) is the answer.
- `heapq.heappush(busy, (start + load[i], server))` : the server's new finish time is simply arrival time plus load.
- The final list comprehension returns every server tied for the maximum count, since the problem asks for all of them, not just one.

Dry Run

Using `k = 3` , `arrival = [0, 1, 2, 3, 4]` , `load = [6, 2, 2, 2, 2]` :

i	arrival	available before	busy before	action	available after	busy after	counts
0	0	[0,1,2]	[]	assign 0	[1,2]	[(6,0)]	[1,0,0]
1	1	[1,2]	[(6,0)]	assign 1	[2]	[(3,1), (6,0)]	[1,1,0]
2	2	[2]	[(3,1), (6,0)]	assign 2	[]	[(3,1), (4,2), (6,0)]	[1,1,1]
3	3	[]	[(3,1), (4,2), (6,0)]	pop (3,1)→avail; search `3%3=0` in [1]→pos=0 off-end? no, pos=0 valid, assign 1	[]	[(4,2), (6,0)]	[1,2,1]
4	4	[]	[(4,2), (6,0)]	pop (4,2)→avail=[2]; search `4%3=1` →pos off-end→wrap to 0→assign 2…			

Following through exactly matches the earlier worked example: server 1 ends with the highest count (2), matching the expected busiest server.

Complexity Analysis

- **Time:** `O(n log k)` for the heap operations across `n` requests (each request does at most a few heap pushes/pops), plus the sorted-list insert/search costs, which are bounded by `k` per operation in the worst case but amortize well in practice given the problem's constraints. This is a massive improvement over brute force's `O(n * k)`.
- **Space:** `O(k)` for the `available` list, `busy` heap, and `count` array.
- **Why correct:** because arrival times are strictly increasing, releasing finished servers "just in time" before each new request is always safe — no future request can affect whether a past job has finished. The search starting at `i % k` with wraparound exactly reproduces the stated routing rule.

Alternative Solutions

For true `O(log k)` lookups at full scale, a balanced order-statistics structure (like a Fenwick tree over server indices, or a `SortedList` from `sortedcontainers`) can replace the plain Python list + `bisect`, since Python list inserts/deletes are `O(k)` due to element shifting. This matters more as `k` approaches 100,000 with many insert/remove cycles; for this problem's given constraints, the simpler list-based approach is fast enough in practice, which the timing comparison below confirms.

Edge Cases

- **All requests on one server:** if `k = 1`, every request either goes to server 0 or gets dropped — the logic still holds since `i % 1` is always `0`.
- **No requests ever dropped:** if load is always short relative to arrival spacing, `available` never empties and the heap release step does most of the work.
- **Every request dropped after the first few:** once all `k` servers are simultaneously busy and no more free up in time, subsequent requests simply increment nothing.
- **Tie for busiest server:** the final list comprehension correctly returns multiple indices if several servers share the maximum count.
- **Maximum scale (`k` and request count both 100,000):** this is where brute force visibly fails and the heap/sorted-list design pays off.

Common Mistakes

1. Using a simple busy/free boolean instead of tracking actual finish times — breaks as soon as a server finishes mid-sequence.
2. Starting the search for a free server at `0` instead of `i % k` — silently violates the problem's exact routing rule.
3. Forgetting to release finished servers *before* searching for an available one for the current request.
4. Forgetting the wraparound: if `bisect_left` returns an index past the end of `available`, failing to fall back to index `0`.
5. Sorting requests by arrival time when they're already strictly increasing — unnecessary and wastes time.
6. Returning only one server index when multiple servers are tied for the maximum.
7. Confusing `heappop` (removes and returns) with peeking `busy[0]` when checking whether a server is actually ready to be released.

Interview Questions

- Why do we start scanning for a free server at `i % k` and not at some other offset?
- Why split available and busy servers into two separate structures instead of one list you filter as you go?
- What's the time complexity if you used a balanced tree instead of a Python list for `available`?
- How would the solution change if requests could arrive out of time order?
- Can you bound the number of heap operations across the whole run?

Similar Problems

- **LeetCode 355 — Design Twitter:** similar pattern of merging multiple time-ordered streams using a heap.
- **LeetCode 253 — Meeting Rooms II:** same free/busy split idea, used to count overlapping intervals.
- **LeetCode 1335 — Minimum Difficulty of a Job Schedule:** different problem, same flavor of "simulate requests against limited resources."
- **LeetCode 1094 — Car Pooling:** another interval/capacity simulation problem that rewards the same "process events in time order" instinct.

Key Takeaways

The lesson here isn't really about servers — it's about recognizing when a problem that reads like "just track state in an array" actually needs you to track *when* state changes. Splitting a resource pool into "ready now" and "busy until some known time," backed by a heap for the busy side, is a pattern that shows up anywhere you simulate things that free up over time: job queues, connection pools, cache evictions, rate limiters. Spotting that pattern — and the one subtle detail about where to start your search — is what separates a brute-force pass from a solution that actually scales.

Watch the Video

For the full walkthrough with the routing trace, the correctness proof, and a real timing comparison between brute force and the optimal solution, watch the video here: {LINK}

About the Series

This breakdown is part of *Fun with Learning Technology*, a series that works through real interview-style coding problems from first principles — not just reciting a solution, but building the intuition that lets you derive it yourself under pressure.

Call To Action

If this walkthrough helped the problem click, give the video a like on YouTube and subscribe for more deep dives like it. For long-form breakdowns delivered straight to your inbox, subscribe to the Substack. Drop a comment with your own approach or a trickier edge case, and share this with anyone prepping for interviews.

Quick Review

Bucket-queue pattern: when you see N workers cycling requests in order, think?

Round-robin assignment with a free-server pool (sorted set/heap) and a busy-until-time map — not brute simulation.

Time complexity of solving busiest-servers efficiently?

O(n log k) using two sorted sets (available/busy) of size k, where n = number of requests.

Space complexity for the server-request assignment approach?

O(k) for the available/busy server sets plus O(k) for a request count array.

Trickiest bug when checking which server is free for request i?

Must search starting from server (i mod k), wrapping circularly — not from server 0 — or assignment order breaks.

This problem vs simple round-robin load balancing?

Unlike plain round-robin, a server gets skipped entirely if busy, so you need fast lookup of the next available server, not just i mod k.

Interview follow-up: why use two sorted containers instead of one array scan per request?

Linear scan for a free server is O(k) per request; sorted sets give O(log k) lookup/removal, avoiding O(n·k) overall.
